AIM

Adaptive Isolating Model Compression Algorithm

Canonical Specification

Jacob Matthew Ritchey

March 11, 2026

This document is the canonical specification for the AIM compression algorithm.
It is sufficient to implement a correct encoder and decoder in any programming language or instruction set without reference to any other source. The reference implementation is aim.py.

1. Overview

AIM (Adaptive Isolating Model) is a lossless data compression algorithm. It operates by recursively decomposing a byte stream into bit-plane components, encoding each component with the information-theoretic model best suited to its structure, and reducing the symbol space at each level. The algorithm is **fully deterministic**, has **no parameters**, and terminates on a **mathematical proof** rather than a heuristic.

AIM achieves compression ratios below raw gzip on audio and other structured binary data by exploiting bit-plane correlations that a byte-level sliding window cannot reach.

Property	Value
Magic bytes	0x41 0x49 0x4D 0x34 (ASCII: AIM4)
Max symbol depth	8 levels (one per bit of a byte)
Halt conditions	3 (structural, not heuristic)
Flag formats	7 (race selects minimum)
Encode modes	2 (Recursive, C-AIM)
Integrity check	SHA-256 over original data
Container overhead	45 bytes fixed

2. Algorithm

AIM consists of five operations that repeat at each level, plus three halting conditions.

2.1 The Five Operations

Operation 1: Entropy Scan (Sweep)

For each bit position b in {0, 1, 2, 3, 4, 5, 6, 7}, count the number of bytes in the current stream that have bit b set. Let k_b denote this count and n the stream length. The flag density for bit b is:

```
d_b = k_b / n
```

Select the bit position with minimum k_b. This is the sparsest bit plane — the one whose information content is furthest from maximum entropy (d = 0.5). Ties are broken by taking the lowest-numbered bit position.

Operation 2: Separation (Bit-Clear)

Let `b*` be the selected bit. Produce two objects:

- **Flag set F**: the sorted list of all byte positions i where bit b* is set in stream[i]. This is a subset of {0, ..., n-1} of size k_{b*}.
- **Aligned stream A**: a copy of the stream with bit b* cleared in every byte. Formally: A[i] = stream[i] AND NOT(1 << b*).

These two objects are lossless: the original stream is exactly recoverable from (A, F, b*) by setting bit b* in A[i] for every i in F.

Operation 3: Flag Set Compression (Race)

Encode F using all applicable format encoders simultaneously. Select the encoder that produces the smallest output. The seven formats and their selection conditions are defined in Section 4.

Operation 4: Symbol Space Reduction (Remap)

The aligned stream A has bit b* cleared in every byte. This means byte values are restricted to the sub-range where bit b* is 0. Remap A to remove the gap: for each byte value v, produce a new value v' by deleting bit b* from v's binary representation and shifting the higher bits down. Formally, for bit position b*:

```
high = v >> (b* + 1)          # bits above b*
low  = v & ((1 << b*) - 1)    # bits below b*
v'   = (high << b*) | low     # pack out the gap
```

After remap, byte values are in the range [0, 2^(7-depth) - 1]. The symbol alphabet has strictly halved. The inverse operation (`unremap`) reinserts a zero bit at position b* and shifts the higher bits up.

Operation 5: Recursion

Apply the five operations to the remapped stream at depth+1. Each level produces one (bit, format, flag_block) tuple. The process continues until a halt condition is reached (see Section 2.2).

2.2 Halt Conditions

There are exactly three halting conditions. Each is decidable in O(n) time. None requires prediction or a threshold parameter.

Code	Name	Condition	Meaning
HALT_ZERO	Perfect decomp.	all(A[i] == 0)	All information was in F. Aligned stream carries no further signal.
HALT_ONE	1-bit floor	all(A[i] == 1)	Stream has reached the 1-bit value floor. No further extraction is meaningful.
HALT_DEPTH	Symbol exhaustion	depth == 8	All 8 bit positions of the byte have been processed. Mathematically unreachable before HALT_ZERO on valid data.

2.3 Why Depth 8 is the Exact Bound

Each remap operation reduces the symbol space by exactly one bit. The progression is:

Depth	Max symbol value	Symbol space
0 (input)	255	8-bit: [0, 255]
1	127	7-bit: [0, 127]
2	63	6-bit: [0, 63]
3	31	5-bit: [0, 31]
4	15	4-bit: [0, 15]
5	7	3-bit: [0, 7]
6	3	2-bit: [0, 3]
7	1	1-bit: {0, 1}
8	0	0-bit: {0} only — HALT_ZERO fires at depth 7

At depth 7 the symbol space is {0, 1}. The sweep selects the bit with fewest set positions. Bit-clear produces an aligned stream of all zeros. HALT_ZERO fires. HALT_DEPTH at depth 8 is a mathematically unreachable safety net for valid byte data.

2.4 Encode Modes

AIM has two encode modes. Both produce identical information content. The mode byte in the container selects how the levels are stored.

Mode	Byte	Description
Recursive (MODE_RECURSIVE)	0	Each level's flag block is encoded separately. The terminal gzip covers only the final aligned stream. Best for non-correlated data.
C-AIM (MODE_C-AIM)	1	All levels' flag bitsets are concatenated before compression. A single LZ77+Huffman pass covers the entire flag sequence, exposing cross-level correlations. Best for structured and audio data. Typically 40x faster than recursive at large file sizes.

The encoder runs both modes, compares total output size, and writes the smaller result. The mode byte in the container header identifies which was chosen.

3. Container Format

Every AIM-compressed file begins with a 45-byte fixed header, followed by the payload.

3.1 Header Layout

`magic[4]`	`mode[1]`	`orig_n[8]`	`sha256[32]`

Total header size: 4 + 1 + 8 + 32 = 45 bytes.

Field	Size	Type	Description
magic	4 B	bytes	0x41 0x49 0x4D 0x34 — ASCII 'AIM4'. Identifies the file format.
mode	1 B	uint8	0 = Recursive, 1 = C-AIM.
orig_n	8 B	uint64 BE	Original uncompressed data length in bytes.
sha256	32 B	bytes	SHA-256 digest of the original uncompressed data. Verified on decode.

3.2 Payload

Immediately following the 45-byte header is the payload. Its structure depends on the mode byte.

- Mode 0 (Recursive): the payload is a chain of level headers followed by a terminal node. See Section 5.
- Mode 1 (C-AIM): the payload is a single C-AIM block. See Section 6.

4. Flag Formats

The flag set F (the sorted list of positions where the selected bit was set) is encoded using one of seven formats. The encoder races all applicable formats and stores the smallest result. A 1-byte format identifier in each level header identifies which format was chosen.

ID	Name	Applicable density	Description
0	GAP	d <= 0.30	VLC-encoded gap list
1	BITSET	any	Raw packed bitset bytes (no entropy coding)
2	EF	d <= 0.15	Elias-Fano
3	RLE	any	Gamma-coded bit-run lengths
4	HUFFMAN	any	Order-0 canonical Huffman on bitset bytes
5	LZ77	0.15-0.85	LZ77 on bitset bytes, no entropy wrapper
6	LZ77HUFF	any	LZ77 + structured Huffman (DEFLATE). Reference impl: gzip level 9.

4.1 FMT_GAP (ID 0) — VLC Gap List

Encodes F as a list of gaps between consecutive positions. Let F = [p_0, p_1, ..., p_{k-1}]. The gap sequence is G = [p_0, p_1-p_0, p_2-p_1, ..., p_{k-1}-p_{k-2}]. All gap values are positive integers in [1, n-1].

Wire format:

```
k_field[4]    uint32 BE — count of positions
                bit 31 = 0: 1-byte gaps  (max gap value 255)
                bit 31 = 1, bit 30 = 0: 2-byte gaps (max 65534)
                bit 31 = 1, bit 30 = 1: 4-byte gaps (max 2^30-1)
gaps[k]       k entries of 1B, 2B, or 4B each (big-endian)
```

If k = 0 (the bit was never set): `k_field = 0x00000000`, no further bytes.

4.2 FMT_BITSET (ID 1) — Raw Bitset

Packs the n flag positions as a bitset of ceil(n/8) bytes. Bit i of byte floor(i/8) is 1 if position i is in F, 0 otherwise. Bit order within each byte is LSB-first: position i maps to bit (i mod 8) of byte (i >> 3).

```
bitset[ceil(n/8)]   packed bits, LSB-first within each byte
```

No length prefix is needed: the decoder knows n from the container header and computes the bitset size as ceil(n/8).

4.3 FMT_EF (ID 2) — Elias-Fano

Encodes F as an Elias-Fano representation. Given n positions and k elements, define l = max(0, floor(log2(n/k))). Each value p in F is split into a lower part (p mod 2^l, stored explicitly) and an upper part (p >> l, stored in unary in a bitvector).

```
  k[4]           uint32 BE — number of set positions
  n[4]           uint32 BE — stream length
  l[1]           uint8  — lower bits per element
  upper_len[4]   uint32 BE — byte length of upper bitvector
  upper[*]       unary-coded upper parts (1 per increment, 0 as separator)
  lower[*]       l bits per element, packed LSB-first, ceil(k*l/8) bytes
```

4.4 FMT_RLE (ID 3) — Gamma-Coded Run Lengths

Treats the n-bit indicator sequence (1 at each position in F, 0 elsewhere) as a sequence of alternating runs of zeros and ones. Encodes each run length using Elias Gamma coding. A leading bit indicates whether the sequence begins with a zero-run or a one-run.

```
  first_is_ones[1]   uint8 — 0 if sequence starts with zeros, 1 if ones
  n_runs[4]          uint32 BE — total number of runs
  gamma_bits[*]      Elias Gamma codes for each run length, packed MSB-
first
```

Elias Gamma encoding of integer x (x >= 1):

- Write floor(log2(x)) zero bits.
- Write a 1 bit.
- Write the x - 2^floor(log2(x)) remaining bits of x in binary (floor(log2(x)) bits).

Example: x=5 = 101 in binary. floor(log2(5))=2. Encoding: 0 0 1 0 1.

4.5 FMT_HUFFMAN (ID 4) — Order-0 Canonical Huffman

Applies canonical Huffman coding to the bitset bytes (the ceil(n/8)-byte packed bitset from FMT_BITSET). Treats the bitset as a sequence of 256-symbol bytes and builds an order-0 Huffman tree from the byte frequency distribution.

```
  lengths[256]    uint8[256] — canonical code length for each byte symbol
0-255
                   0 = symbol does not appear
                   1-15 = code length in bits
  n_bits[4]       uint32 BE — total number of encoded bits
  packed[*]       ceil(n_bits/8) bytes, bits packed MSB-first
```

Canonical code assignment:
Sort symbols by (length, symbol_value) ascending. Assign codes in order: the first symbol of length L gets code 0, each subsequent symbol of the same length increments the code by 1, and on length increase the code is left-shifted by the length difference.

4.6 FMT_LZ77 (ID 5) — LZ77

Applies LZ77 to the bitset bytes using a hash-chain search with a 32768-byte sliding window. Produces a token stream. No entropy coding is applied to the tokens.

```
orig_n[4]       uint32 BE — original bitset byte count
n_tokens[4]     uint32 BE — total token count
token groups:
  flag[1]         1 byte per 8 tokens, MSB = first token
                  bit = 1: match token follows
                  bit = 0: literal token follows
  per match:      dist[2] uint16 BE + (length-3)[1] uint8
  per literal:    byte[1] uint8
```

Minimum match length: 3. Maximum match length: 258. Maximum distance: 32768. The match length is stored as (length - 3) to fit in one byte.

4.7 FMT_LZ77HUFF (ID 6) — LZ77 + Structured Huffman

LZ77 followed by a structured Huffman entropy coder applied to the match token stream. The reference implementation uses `gzip(level=9)` (RFC 1952 / DEFLATE, RFC 1951), which is precisely this combination with length-distance Huffman codes. Decoders must accept any DEFLATE-compliant output as the payload for this format.

```
payload[*]      DEFLATE-compressed bitset bytes (RFC 1951)
                wrapped in gzip container (RFC 1952)
```

Note on format ID 6
FMT_LZ77HUFF is architecturally defined as LZ77 + structured Huffman coding of the length-distance token stream. The gzip/DEFLATE standard is the canonical realization of this combination. Implementations may use any DEFLATE-compatible library.
The output is interoperable as long as decompression produces the original bitset bytes.

5. Recursive Mode Payload (Mode 0)

In recursive mode, the payload is a chain of level nodes. Each node except the innermost has a fixed 11-byte header. The innermost node is a terminal.

5.1 Level Node Header

`bit[1]`	`fmt[1]`	`flag_len[4]`	`halt[1]`	`child_len[4]`	`flag_data[flag_len]`

Field	Size	Type	Description
bit	1 B	uint8	The bit position selected by the sweep (0-7).
fmt	1 B	uint8	Flag format ID (0-6). See Section 4.
flag_len	4 B	uint32 BE	Byte length of the flag_data block.
halt	1 B	uint8	Halt code (see below).
child_len	4 B	uint32 BE	Byte length of the child block that follows flag_data.
flag_data	flag_len B	bytes	Encoded flag set. Format determined by fmt.
child	child_len B	bytes	Child payload (next level header, or terminal data).

5.2 Halt Codes

Value	Name	Meaning	Child contents
0	HALT_RECURSE	Continue recursing	Next level node header
1	HALT_TERMINAL	Terminal with flags	LZ77+Huffman of final aligned stream
2	HALT_ZERO	Aligned stream all zeros	Empty (child_len = 0)
3	HALT_ONE	Aligned stream all ones	Empty (child_len = 0)

When halt = HALT_ZERO or HALT_ONE, flag_len may be 0 (pure terminal, no flags to apply) or non-zero (a cake-check halt preserved for compatibility). If flag_len > 0, apply the flags before returning the halt result.

5.3 Recursive Encode Procedure

The encoder is iterative. It collects level tuples and assembles bottom-up.

```
levels = []
current = data

for depth in range(MAX_DEPTH=8):
    if is_all_zero(current) or is_all_one(current): break
    b* = sweep(current)
    A, F = bit_clear(current, b*)
    flag_block, fmt = race(F, len(current))
    levels.append( (b*, fmt, flag_block) )
    current = remap(A, b*)

# Build terminal
if is_all_zero(current): inner = HALT_ZERO node
elif is_all_one(current): inner = HALT_ONE node
else: inner = HALT_TERMINAL node with lz77huff(current)

# Assemble bottom-up
payload = inner
for (b*, fmt, flag_block) in reversed(levels):
    payload = level_header(b*, fmt, flag_block, HALT_RECURSE, payload)
```

5.4 Recursive Decode Procedure

The decoder is iterative. It walks level headers forward, then rebuilds bottom-up.

```
levels = []
pos = 0

loop:
    read 11-byte header at pos
    read flag_data[flag_len] at pos+11
    pos += 11 + flag_len

    if halt == HALT_RECURSE:
        levels.append( (bit, fmt, flag_data) )
        # continue loop
    elif halt == HALT_ZERO:  current = bytes(n, 0x00); break
    elif halt == HALT_ONE:   current = bytes(n, 0x01); break
    elif halt == HALT_TERMINAL:
        current = lz77huff_decompress(child)
        if flag_len > 0: levels.append( (bit, fmt, flag_data) )
        break

# Rebuild bottom-up
for (bit, fmt, flag_data) in reversed(levels):
    aligned  = unremap(current, bit)
    flag_pos = decode_fmt(flag_data, n, fmt)
    current  = reconstruct(aligned, flag_pos, bit)
return current
```

6. C-AIM Mode Payload (Mode 1)

C-AIM (Concatenated AIM) does not remap between levels. All levels clear bits from the same **full [0, 255] symbol space**. Each level's flag bitset is collected. All bitsets are concatenated and compressed in a single LZ77+Huffman pass, exposing cross-level correlations.

6.1 C-AIM Payload Layout

`n_levels[1]`	`bits_hdr[n_levels]`	`flags_len[4]`	`flags_gz[flags_len]`	`term_len[4]`	`term_gz[term_len]`

Field	Size	Type	Description
n_levels	1 B	uint8	Number of levels processed.
bits_hdr	n_levels B	uint8[]	Bit position selected at each level, in depth order.
flags_len	4 B	uint32 BE	Byte length of the compressed flag concatenation.
flags_gz	flags_len B	bytes	LZ77+Huffman(concat of all flag bitsets in depth order). Each bitset is ceil(n/8) bytes using the LSB-first packing from FMT_BITSET (Section 4.2).
term_len	4 B	uint32 BE	Byte length of the compressed terminal.
term_gz	term_len B	bytes	LZ77+Huffman(final aligned stream after all levels).

6.2 C-AIM Encode Procedure

```
bitsets = []
which_bits = []
cleared = 0   # bitmask of already-used bit positions
current = data

for depth in range(MAX_DEPTH=8):
    avail = [b for b in 0..7 if not cleared[b]]
    if not avail: break
    b* = argmin over avail of count_set_bits(current, b)
    A, F = bit_clear(current, b*)
    bitsets.append( build_bitset(F, n) )  # ceil(n/8) bytes
    which_bits.append(b*)
    cleared |= (1 << b*)
    current = A   # no remap in C-AIM
    if is_all_zero(current) or is_all_one(current): break

flags_gz = lz77huff( concat(bitsets) )
term_gz  = lz77huff( current )

payload = [n_levels] + which_bits + flags_len + flags_gz
        + term_len + term_gz
```

6.3 C-AIM Decode Procedure

```
n_levels = payload[0]
which_bits = payload[1 : 1 + n_levels]
flags_len = uint32_be(payload, 1 + n_levels)
flags_gz  = payload[5 + n_levels : 5 + n_levels + flags_len]
term_len  = uint32_be(payload, 5 + n_levels + flags_len)
term_gz   = payload[9 + n_levels + flags_len :]

flags_cat = lz77huff_decompress(flags_gz)  # n_levels * ceil(n/8) bytes
current   = lz77huff_decompress(term_gz)

bs_size = ceil(n / 8)

for i in range(n_levels - 1, -1, -1):  # reverse order
    b*      = which_bits[i]
    bitset  = flags_cat[i * bs_size : (i+1) * bs_size]
    F       = bitset_to_positions(bitset, n)
    current = reconstruct(current, F, b*)
    # no unremap — C-AIM does not remap

return current
```

Note: C-AIM does not call unremap because it does not call remap. The bit positions in `which_bits` are cleared from the original symbol space directly, so reconstruction is a simple OR of the flag positions into the aligned stream.

7. Core Operation Definitions

7.1 build_bitset(F, n)

Returns a ceil(n/8)-byte buffer. For each position p in F, sets bit (p mod 8) of byte (p >> 3). All other bits are 0.

```
buf = bytearray(ceil(n / 8))
for p in F:
    buf[p >> 3] |= (1 << (p & 7))
return bytes(buf)
```

7.2 bitset_to_positions(bitset, n)

Inverse of build_bitset. Returns the sorted list of set positions.

```
return [i for i in range(n) if (bitset[i >> 3] >> (i & 7)) & 1]
```

7.3 remap(A, b*)

Pack out the cleared bit slot. For each byte v in A:

```
high = v >> (b* + 1)
low  = v &  ((1 << b*) - 1)
v'   = (high << b*) | low
```

7.4 unremap(R, b*)

Reinsert a zero bit at position b*. For each byte v in R:

```
high = v >> b*
low  = v &  ((1 << b*) - 1)
v'   = (high << (b* + 1)) | low
```

7.5 reconstruct(A, F, b*)

Restore the original stream from the aligned stream and flag positions. Sets bit b* in A[i] for every i in F.

```
result = bytearray(A)
for p in F:
    result[p] |= (1 << b*)
return bytes(result)
```

7.6 sweep(data)

Returns the bit position with minimum set-bit count.

```
counts = [0] * 8
for byte in data:
    for b in range(8):
        if (byte >> b) & 1: counts[b] += 1
return argmin(counts)   # lowest index on tie
```

Optimized with numpy: popcount via bit masking over the full array in one pass.

7.7 is_all_zero(data) / is_all_one(data)

```
is_all_zero(data): return all(b == 0 for b in data)
is_all_one(data):  return all(b == 1 for b in data)
```

8. Format Selection (The Race)

The encoder evaluates all applicable formats for the current flag density and stores the smallest result. The format ID stored in the level header identifies the winning format to the decoder.

Format	Applied when	Notes
FMT_GAP (0)	density <= 0.30	Position list materialized; skipped at high density to avoid O(k) memory.
FMT_BITSET (1)	always	Raw fallback. Never wins in practice but always competes.
FMT_EF (2)	density <= 0.15	Optimal for sparse uniform distributions.
FMT_RLE (3)	always	Wins for bursty bit planes regardless of density.
FMT_HUFFMAN (4)	always	Wins when byte-value entropy dominates.
FMT_LZ77 (5)	0.15 <= density <= 0.85	LZ77 alone. Wins rarely but included for completeness.
FMT_LZ77HUFF (6)	always	Wins for spatially-correlated planes and C-AIM flag concatenations.

9. Integrity Verification

The container header includes a SHA-256 digest of the original uncompressed data. Decoders must compute the SHA-256 of the decoded result and compare it to the header value before returning the data.

```
expected = header.sha256
actual   = sha256(decoded_data)
if actual != expected: raise IntegrityError
```

Verification may be disabled for performance in trusted contexts, but must be the default behavior.

10. Reference Results

Measured on the reference Python implementation (aim_v14.py) against raw gzip level 9.

File	Size	gzip ratio	AIM ratio	vs gzip	Mode
sample-full.wav (PCM audio)	38.7 MiB	100.00%	84.97%	-15.03%	Recursive
sample-full.wav (PCM audio)	38.7 MiB	100.00%	85.06%	-14.94%	C-AIM (2.7s)
Synthetic image-like	256 KB	100.00%	~108%	+8%	C-AIM
Synthetic ramp	100 KB	100.00%	100.0%	~0%	C-AIM
Random data	8 KB	100.00%	100.2%	+0.2%	C-AIM

AIM delivers meaningful compression gains over gzip on data with bit-plane structure (PCM audio, sensor data, structured binary formats). On purely random data it is near-parity with gzip. On image-like spatial data with cross-row byte correlation, gzip retains an advantage because DEFLATE's length-distance Huffman coding compresses approximate row matches more efficiently than the AIM sweep structure.

11. Implementation Notes

11.1 Memory

Both the recursive encoder and decoder are iterative, not recursive. Peak memory is O(n): only one copy of the working data is live at any time. Flag blocks are small (compressed bitsets) and accumulate at O(n/8 * depth) in the worst case.

11.2 Threading

The flag race (Section 8) evaluates up to 7 formats independently. These can be dispatched to a thread pool. For n >= 512 KB, the main thread computes the next level's work while format workers run (scatter-gather pipeline). gzip releases the GIL in CPython, making FMT_LZ77HUFF and FMT_HUFFMAN genuinely parallel.

11.3 Numpy Acceleration

The sweep (Section 7.6) and bit_clear operations benefit substantially from numpy vectorization. The sweep can be computed in a single pass over the byte array using bitwise masking and np.count_nonzero. The bit_clear aligned stream is np.bitwise_and(arr, ~(1 << b*)).

11.4 C Implementation Target

The algorithm maps cleanly to C. The iterative encode and decode structures are explicit stack loops over fixed-size frame arrays. The sweep is a tight inner loop over a byte array. No dynamic memory allocation is required during the sweep, bit-clear, or remap operations. The flag race can use POSIX threads or a work-stealing pool. The reference Python implementation (aim.py) is the normative source for edge-case behavior.

11.5 Edge Cases

Input	Expected behavior
Empty (n=0)	Valid container. Recursive mode. Terminal: HALT_TERMINAL with huffman(empty).
All-zero bytes	Depth 0 sweep finds bit 0, density 0. HALT_ZERO fires immediately.
All-0xFF bytes	Any bit has density 1.0. HALT_ONE fires at depth 0.
Single byte	Standard encode. HALT_ZERO or HALT_ONE likely at depth 0.
Random data	All formats compete; LZ77HUFF wins at bitset density ~50%. Output near raw size.